

Achieving High Availability - Fault Tolerance

In this lesson, you will learn about fault tolerance and designing a HA fault-tolerant service.

We'll cover the following

- What is fault tolerance?
- A highly available fault-tolerant service – Architecture

There are several approaches to achieve *HA*. The most important of them is to make the system fault-tolerant.

What is fault tolerance?

Fault tolerance is a system's ability to stay up despite taking hits.

A *fault-tolerant* system is equipped to handle faults. Being *fault-tolerant* is an essential element in designing life-critical systems.

In a fault-tolerant system, out of several *instances/nodes* running a service, a few go offline and bounce back all the time. In case of these internal failures, the system can work at a reduced level without going down entirely.

An elementary example of a system like this is social networking platforms. In the case of backend node failures, a few services of the app, such as image upload, post likes, etc., may stop working. However, the application as a whole will still be up. This approach technically is also known as *fail soft*.

A highly available fault-tolerant service – Architecture

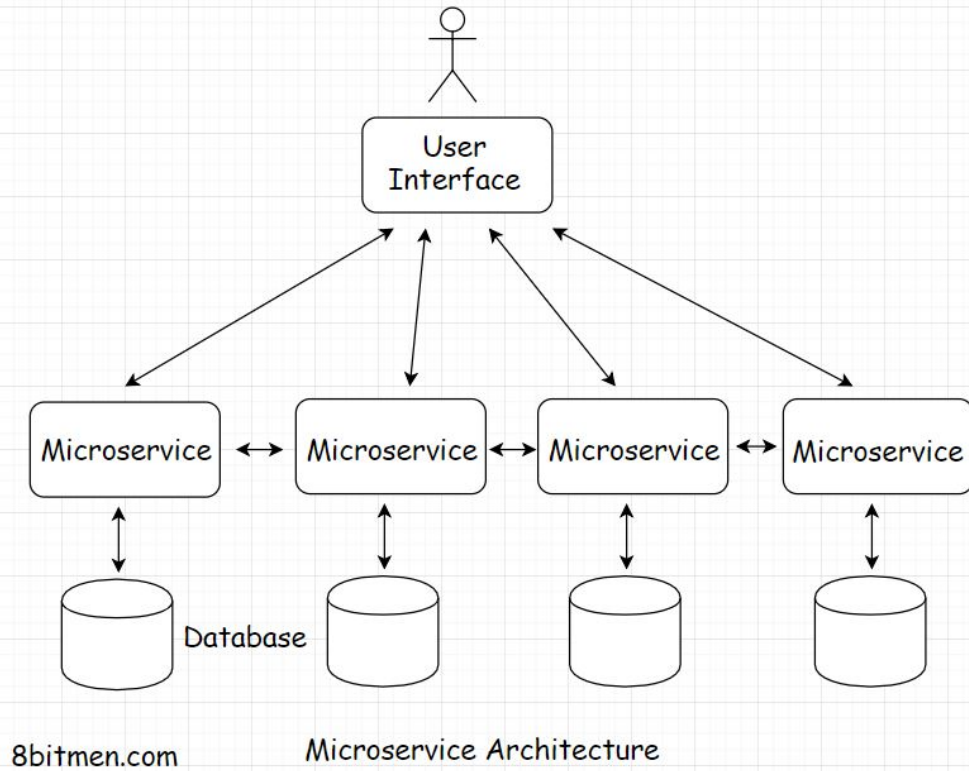
> A *fault-tolerant* system can be designed at the *application* and the *deployment level*. Since apps are typically deployed on the cloud in the modern web landscape, I discuss the deployment infrastructure in detail in my [cloud computing course](#). It would be overkill to state everything in this one.

Also, this course, Web Application and Software Architecture 101, is part one of my [Zero to Software Architect learning track](#), which enables you to be a pro at designing large-scale distributed services from having zero knowledge in the space. The Cloud Computing 101 course follows this course in the learning track; followed by the distributed systems design course.

 **educative**

So, to achieve high availability at the application level, the entire massive service is architecturally broken down into more granular loosely coupled services called *microservices*.





There are many upsides of splitting a big monolith into several microservices:

- Easy management and maintenance
- Ease of development
- Ease of adding new features to a service without affecting other services
- Scalability and high availability of the system

